

Modul 4

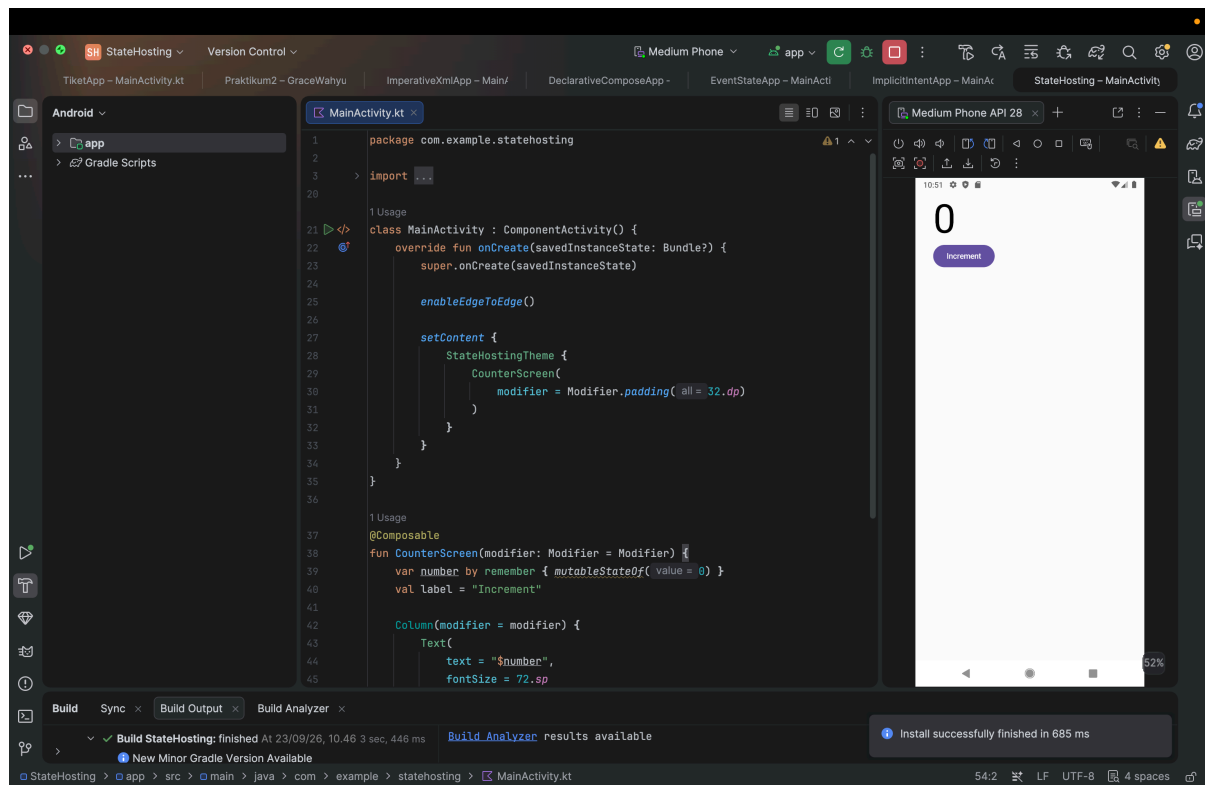
State Hoisting Perubahan Konfigurasi dan Side Effect

Nama	Grace Wahyuni
NIM	245150401111029
Mata Kuliah	Pengembangan Aplikasi Mobile
Dosen Pengampu	Muhammad Aminul Akbar, S.Kom., M.T.

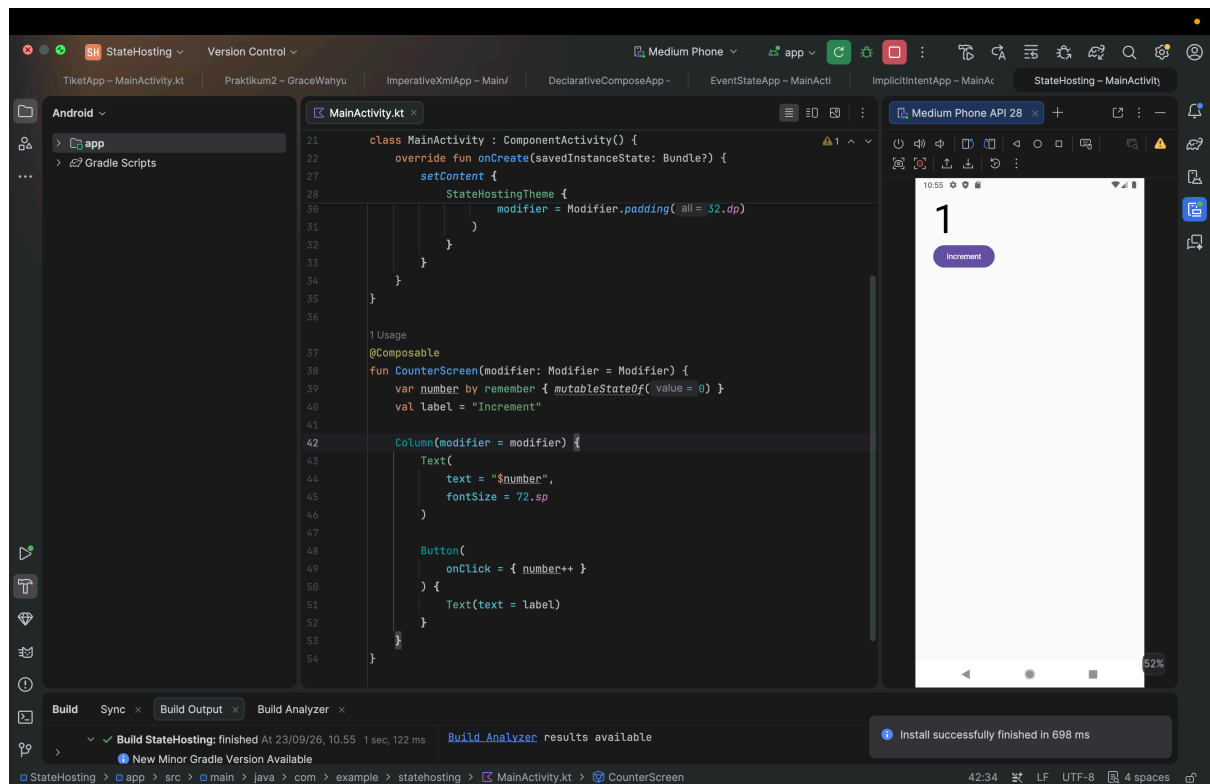
Jawaban

A. State Hoisting

Percobaan ini menunjukkan pemisahan antara tampilan dan pengelolaan state pada Jetpack Compose. Pada tahap awal, number dan label disimpan di dalam CounterScreen. Setelah state hoisting, state dikelola oleh composable induk dan CounterScreen menerima nilai serta callback sebagai parameter.

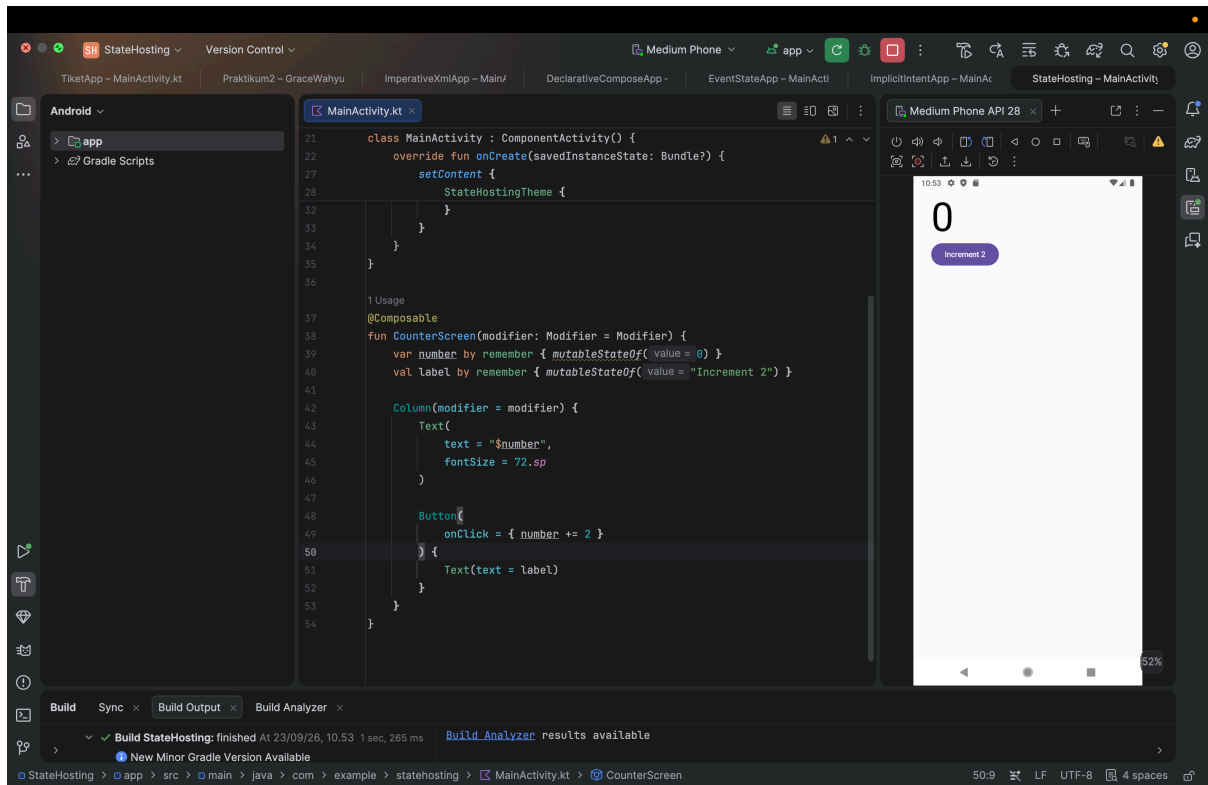


Gambar 1. Kondisi awal aplikasi dengan nilai number = 0 dan tombol Increment.

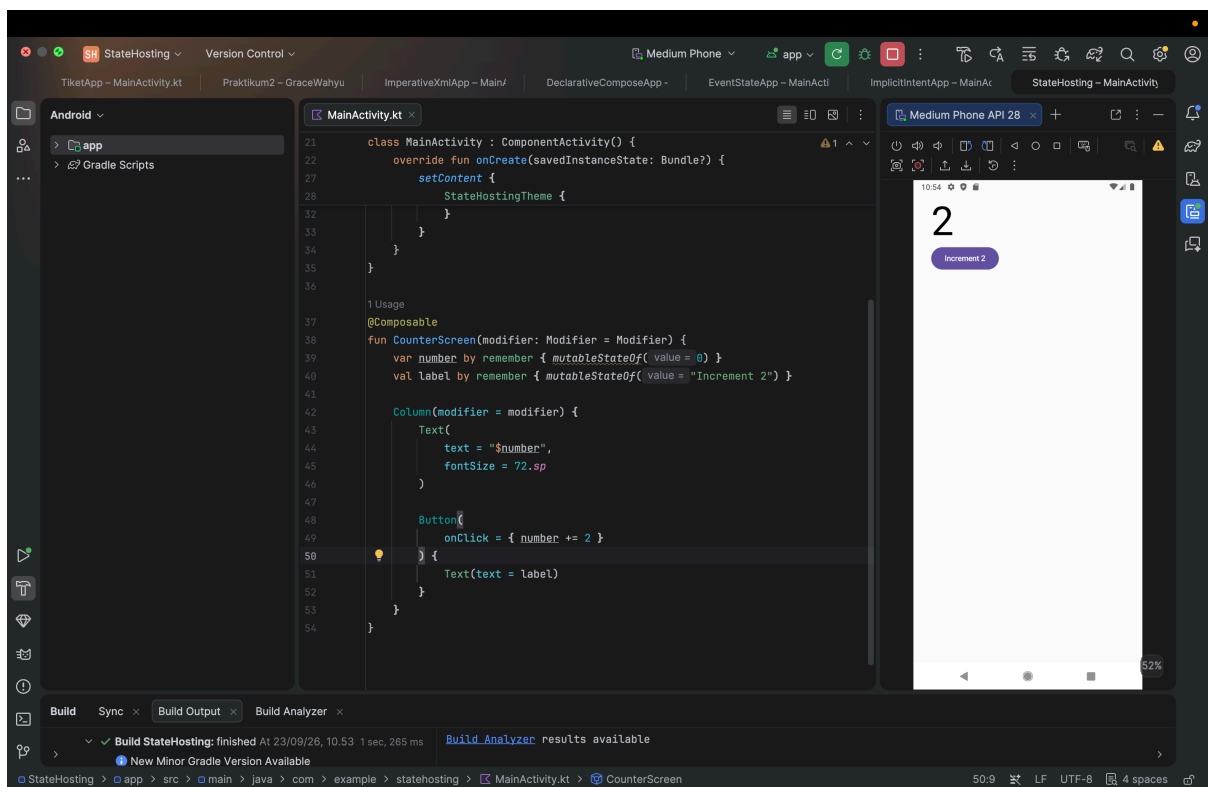


Gambar 2. Nilai number bertambah menjadi 1 setelah tombol Increment diklik.

- **Apa yang terjadi setiap kali tombol Increment diklik?**
 Nilai number bertambah satu. Karena Text membaca number, Compose melakukan recomposition dan menampilkan nilai terbaru pada layar.
- **Apa penyebab perubahan angka pada layar?**
 Callback `onClick` mengubah `mutableStateOf`. Compose mengamati perubahan state tersebut dan menyusun ulang bagian UI yang bergantung pada state.
- **State apa yang digunakan oleh Text dan Button?**
 Text angka menggunakan state `number`, sedangkan label Button menggunakan state `label`. Pada versi awal keduanya dikelola oleh `CounterScreen`.



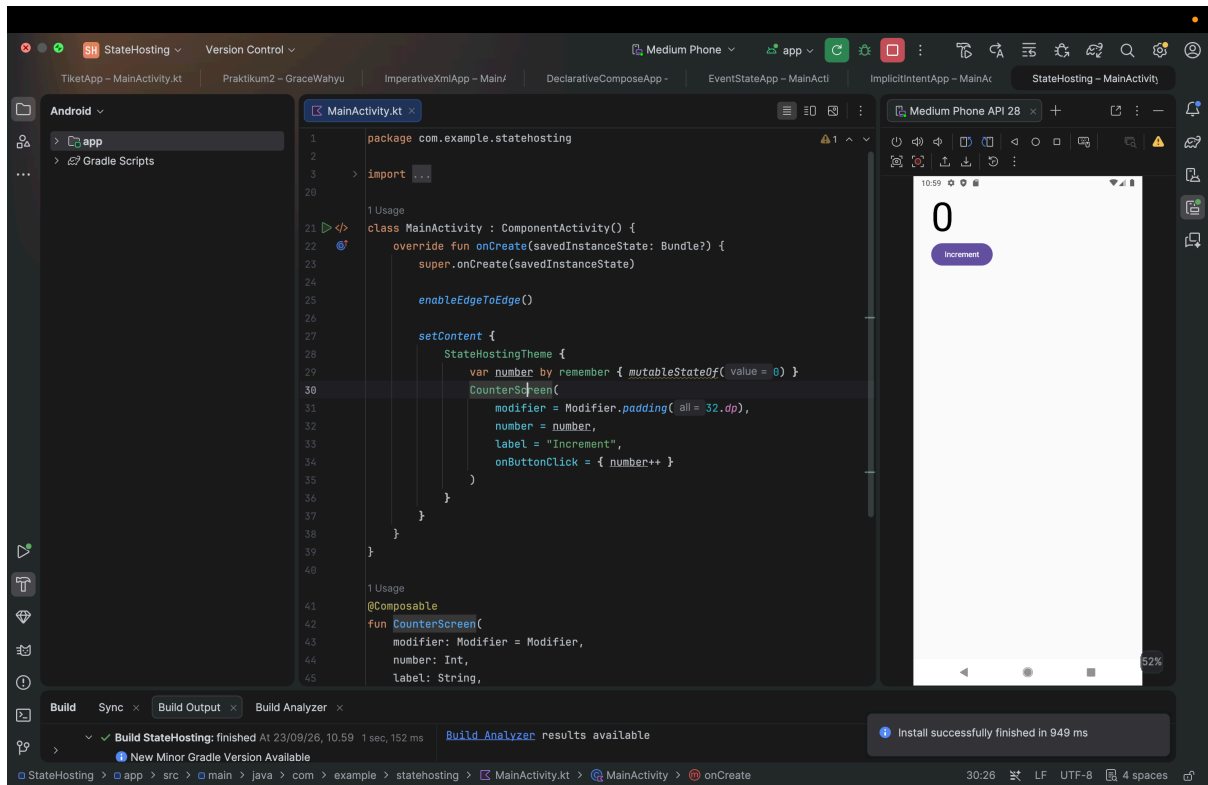
Gambar 3. Kondisi awal percobaan Increment 2 dengan nilai number = 0.



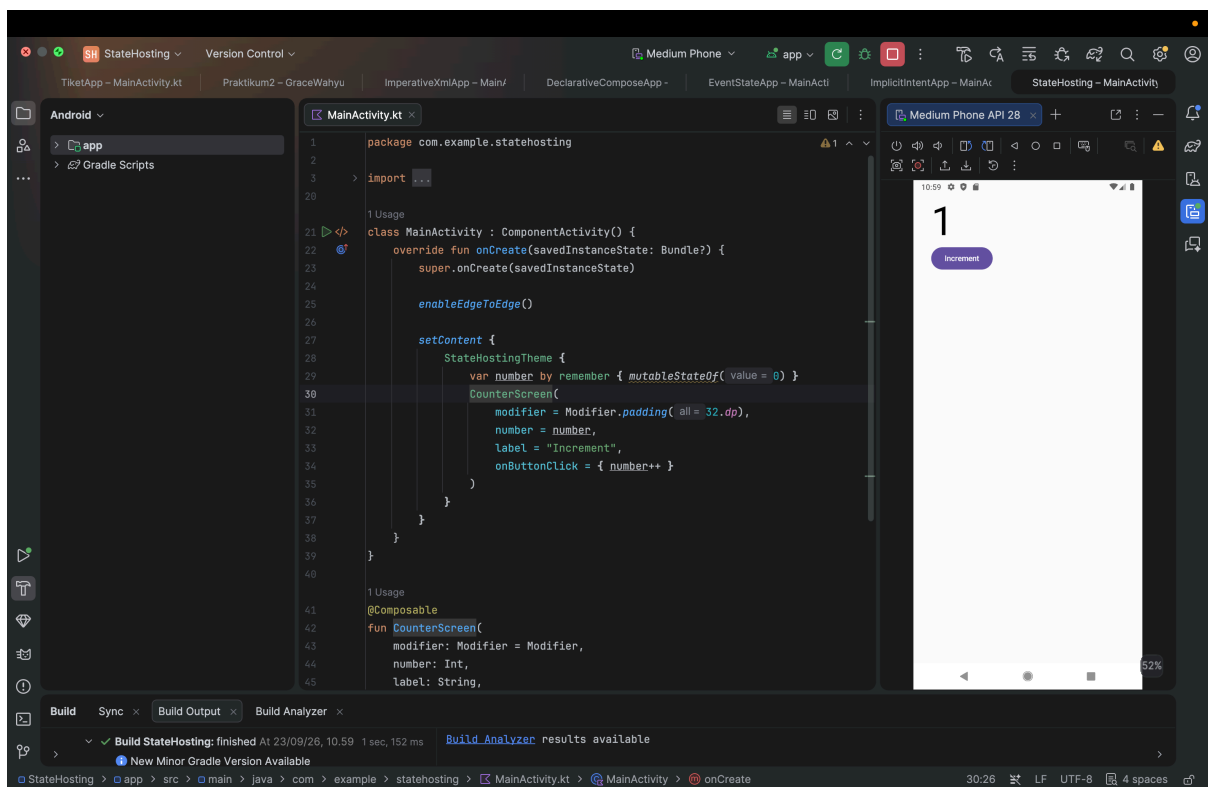
Gambar 4. Nilai number bertambah dua setelah tombol Increment 2 diklik.

- **Perubahan apa yang dilakukan agar angka bertambah dua?**

Callback diubah dari `number++` menjadi `number += 2`, sedangkan label tombol diubah menjadi Increment 2. Pada tahap hoisting, `CounterScreen` dibuat stateless. State dipindahkan ke pemanggil sehingga aliran data satu arah: nilai turun melalui parameter, sedangkan event naik melalui lambda callback.



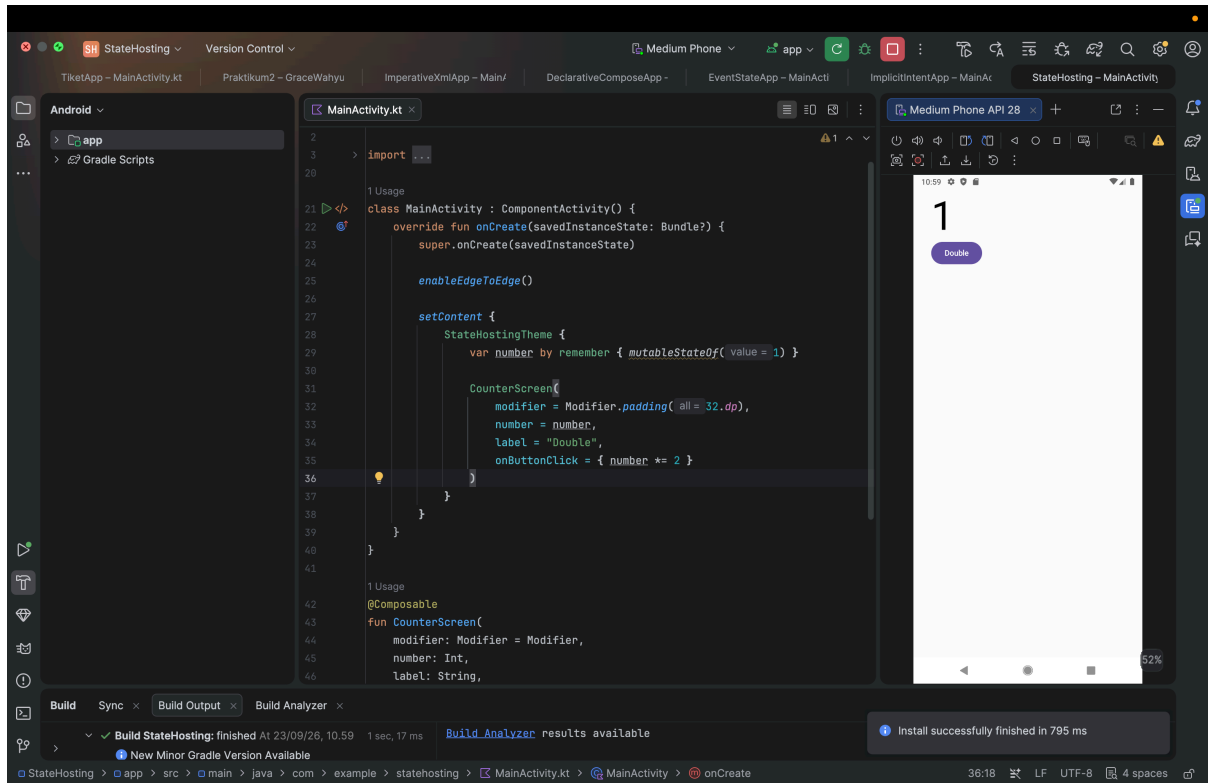
Gambar 5. State number dideklarasikan di pemanggil CounterScreen dan diteruskan sebagai parameter.



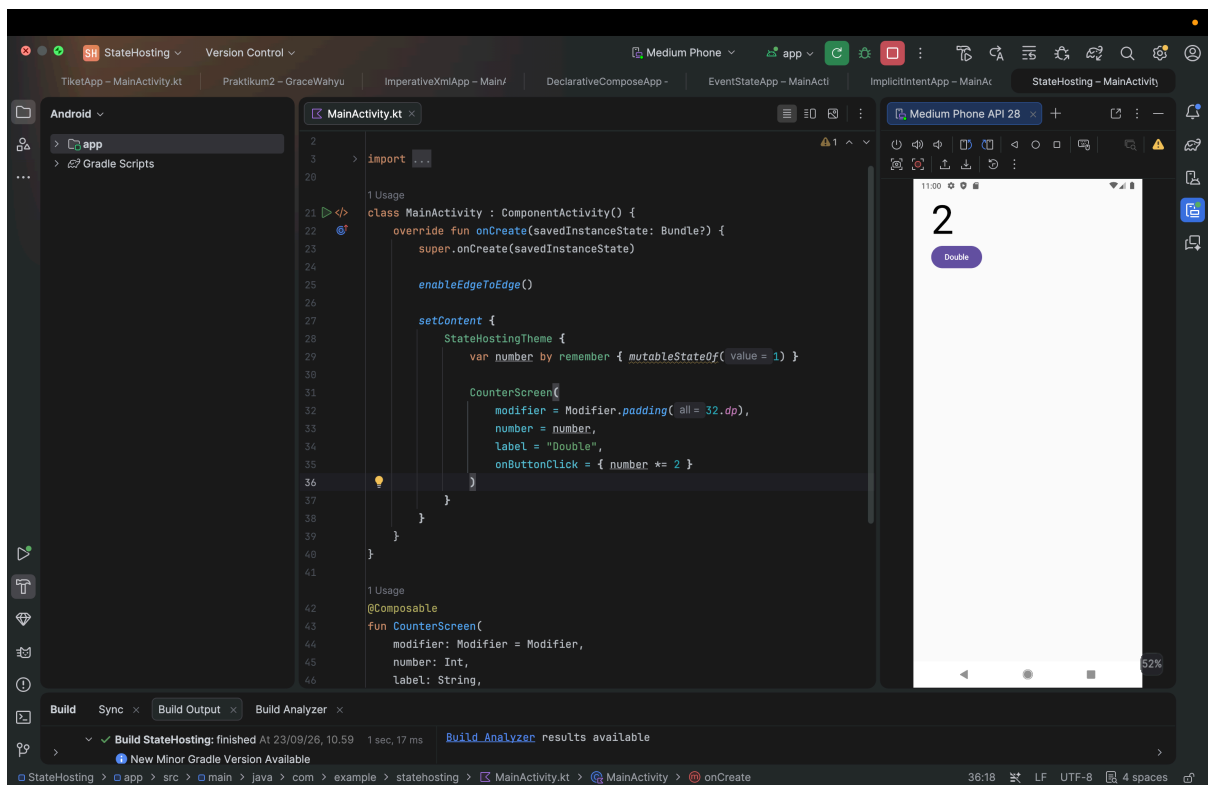
Gambar 6. CounterScreen menampilkan nilai dari state yang sudah dihoist.

- **Apakah program tetap berjalan sama setelah state hoisting?**
Ya. Tampilan dan perilaku tombol tetap sama. Perbedaannya adalah state sekarang berada di composable induk, bukan di CounterScreen.
- **Di bagian mana proses increment dilakukan setelah hoisting?**
Proses increment dilakukan di lambda onClick pada saat CounterScreen dipanggil.
- **Apa manfaat state hoisting ketika kebutuhan berubah?**

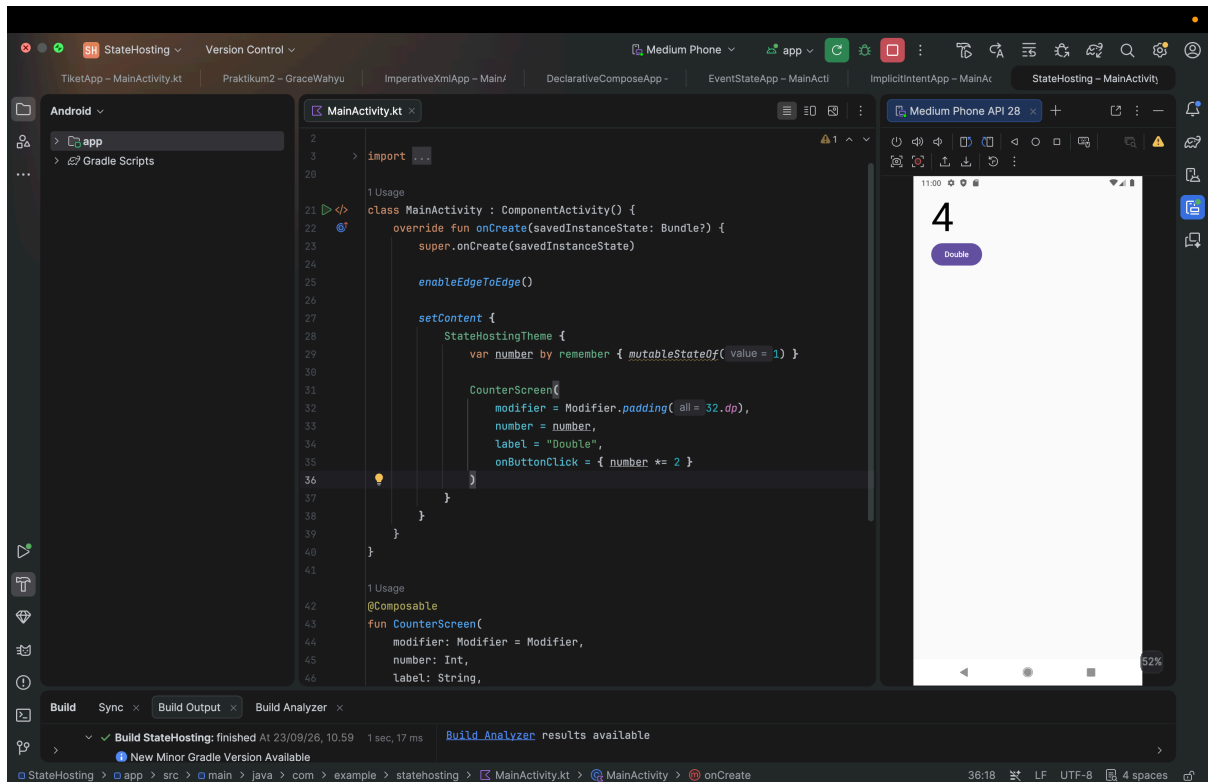
Perubahan label atau logika tombol cukup dilakukan pada pemanggil. Komponen CounterScreen tidak perlu diubah sehingga lebih reusable dan lebih mudah diuji.



Gambar 7. Kondisi awal fungsi Double dengan nilai number = 1.



Gambar 8. Nilai number menjadi 2 setelah satu kali penggandaan.



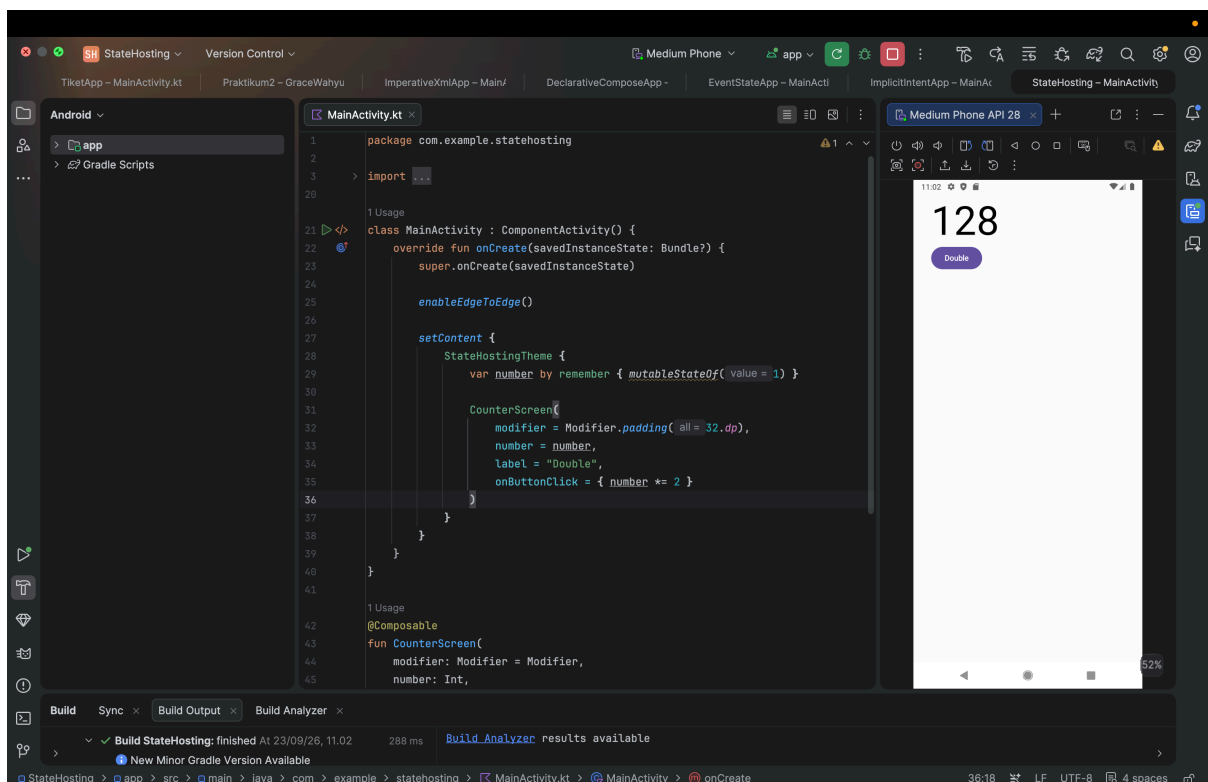
Gambar 9. Nilai number menjadi 4 setelah penggandaan berikutnya.

- Bagaimana perubahan dari Increment menjadi Double dilakukan?

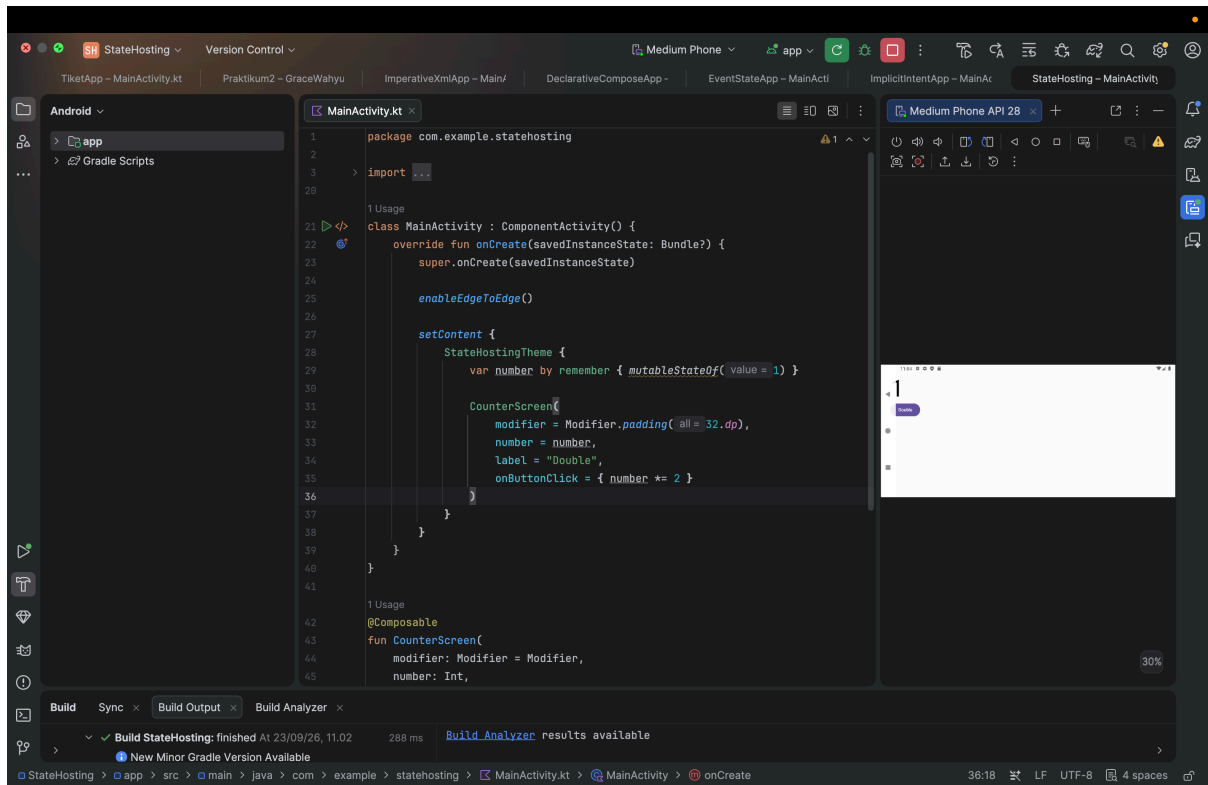
Nilai awal diubah menjadi 1, label menjadi Double, dan callback diubah menjadi $\text{number} = 2$. Tidak ada perubahan pada isi CounterScreen karena logika disimpan pada pemanggil.

B. Perubahan Konfigurasi

Perubahan konfigurasi diuji dengan memutar emulator dari portrait ke landscape. Perputaran menyebabkan Activity dibuat ulang sehingga cara penyimpanan state menentukan apakah nilai number dapat dipulihkan.



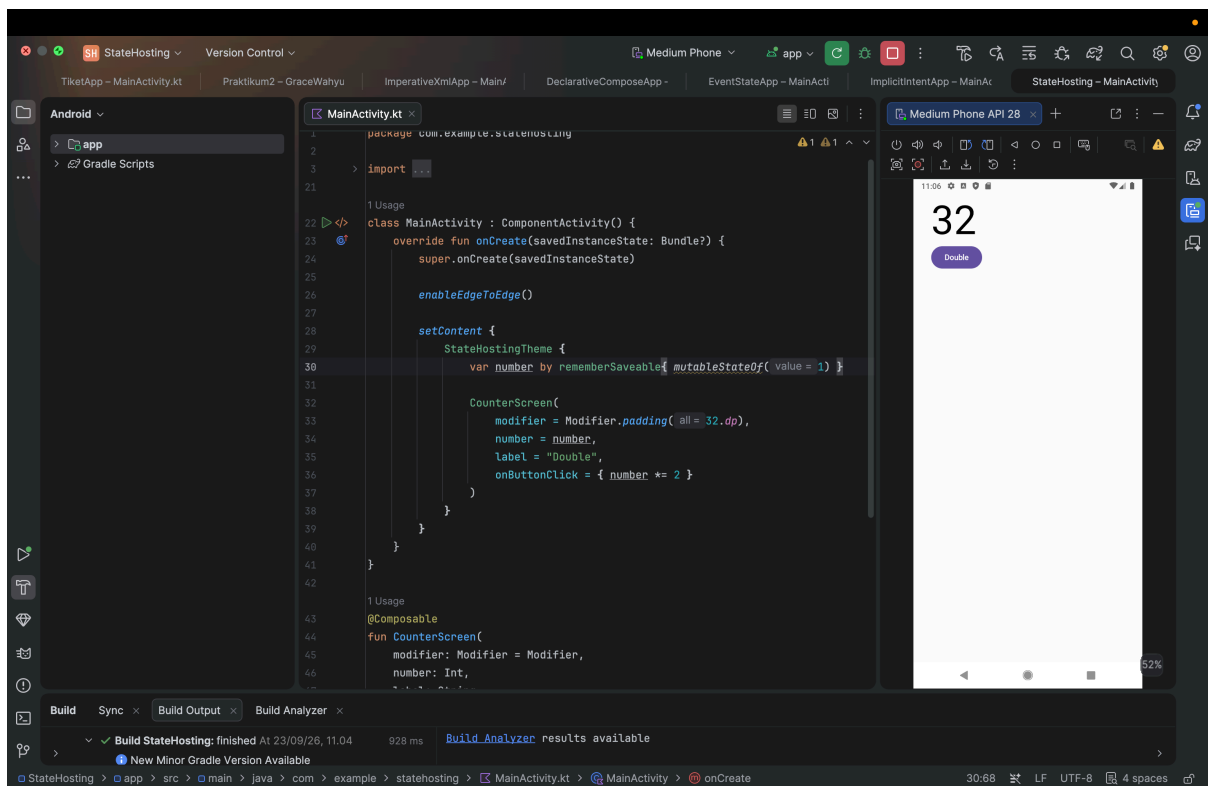
Gambar 10. Nilai number = 128 sebelum rotasi ketika state menggunakan remember.



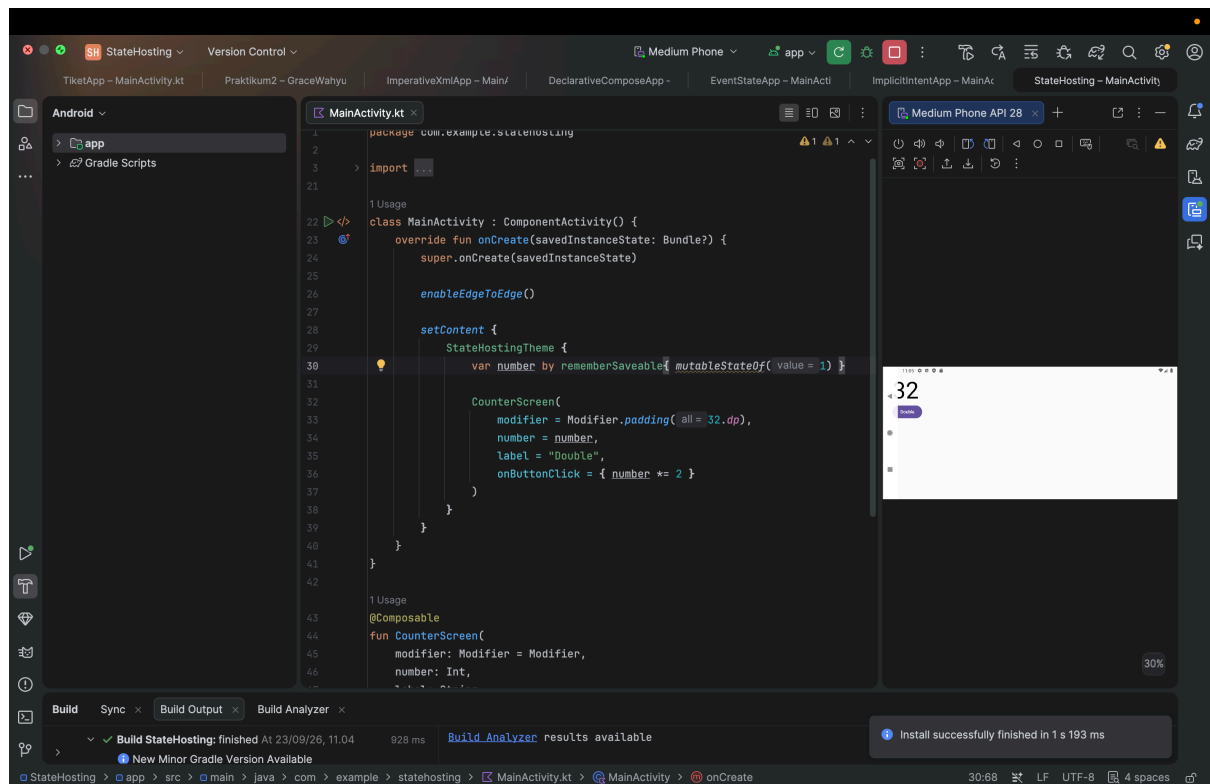
Gambar 11. Tampilan landscape setelah rotasi saat state menggunakan remember; nilai kembali ke nilai awal.

- **Mengapa nilai number berubah setelah rotasi dengan remember?**

Remember hanya mempertahankan state selama composition yang sama. Ketika Activity dibuat ulang akibat rotasi, state dibuat kembali dari nilai awal.



Gambar 12. Nilai number = 32 sebelum rotasi ketika state menggunakan rememberSaveable.



Gambar 13. Nilai tetap dipulihkan setelah rotasi landscape dengan `rememberSaveable`.

- **Apa fungsi `rememberSaveable`?**

Fungsi `rememberSaveable` menyimpan state sederhana melalui saved instance state sehingga nilainya dapat dipulihkan ketika Activity dibuat ulang karena perubahan konfigurasi.

- **Apa perbedaan `remember` dan `rememberSaveable`?**

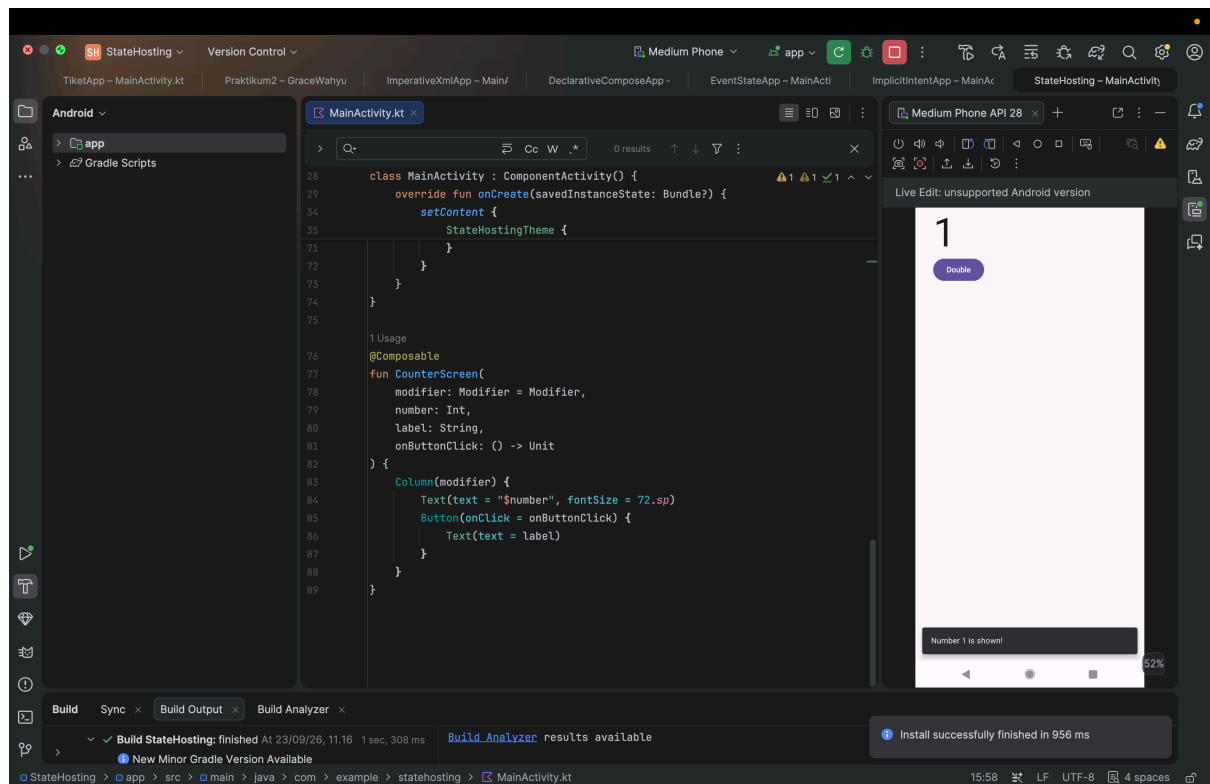
Keduanya bertahan saat proses rekomposisi. `remember` tidak dirancang untuk mempertahankan nilai ketika Activity direkonstruksi, sedangkan `rememberSaveable` dapat memulihkan nilai yang dapat disimpan dalam Bundle.

- **Sebutkan contoh penggunaan `rememberSaveable`.**

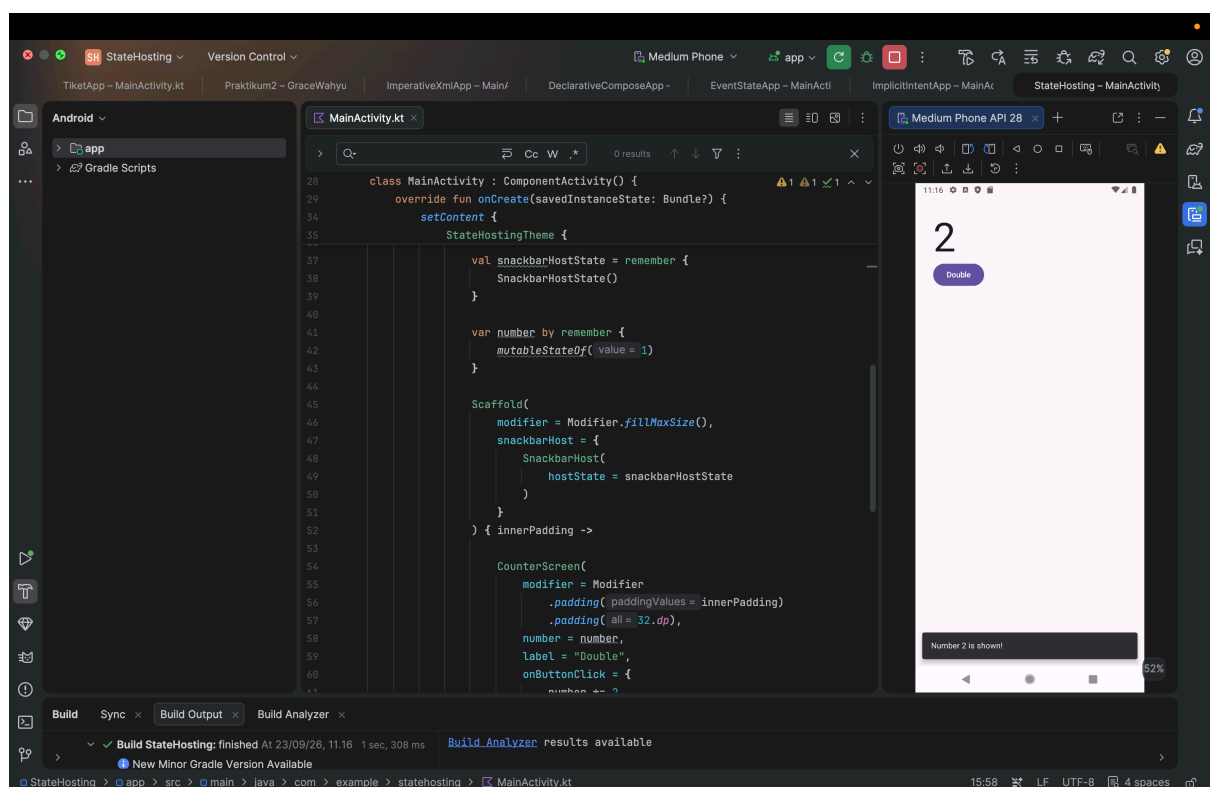
Isi TextField pada form, status checkbox, tab yang dipilih, dan posisi scroll merupakan contoh state UI ringan yang tepat. Data bisnis atau objek kompleks lebih sesuai dikelola ViewModel atau penyimpanan persisten.

C. LaunchedEffect dan Snackbar

Pada percobaan ini, `SnackbarHostState` digunakan sebagai state host Snackbar. `LaunchedEffect(number)` ditempatkan pada scope composable dan dipicu oleh perubahan nilai number.



Gambar 14. Snackbar menampilkan Number 1 is shown saat aplikasi pertama kali masuk composition.



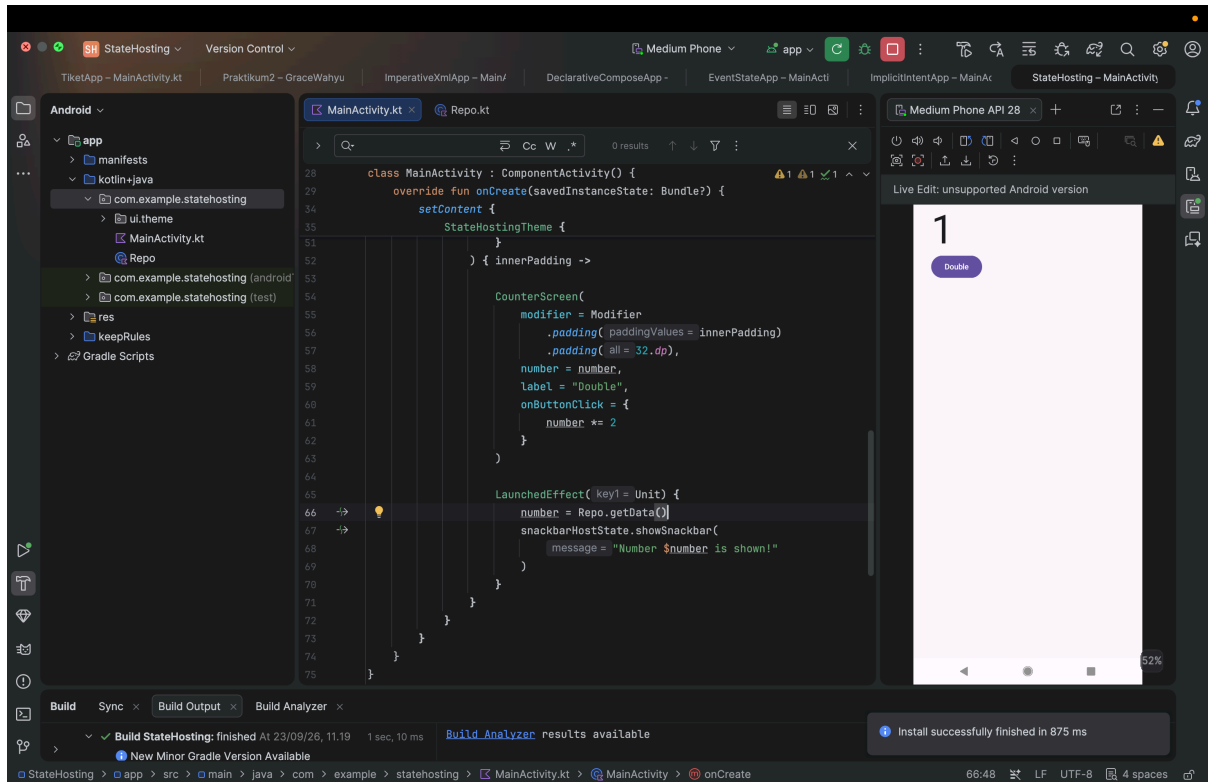
Gambar 15. Snackbar kembali tampil setelah number berubah menjadi 2.

- **Mengapa Snackbar tampil tanpa tombol diklik?**
LaunchedEffect(number) dijalankan saat composible pertama kali masuk composition dengan number bernilai 1. Blok tersebut memanggil showSnackbar.
- **Apa yang memicu Snackbar tampil kembali?**
Perubahan nilai number mengubah key LaunchedEffect(number). Efek sebelumnya dibatalkan jika masih berjalan, lalu efek dijalankan kembali untuk nilai terbaru.
- **Apa yang terjadi setelah tombol Double diklik?**

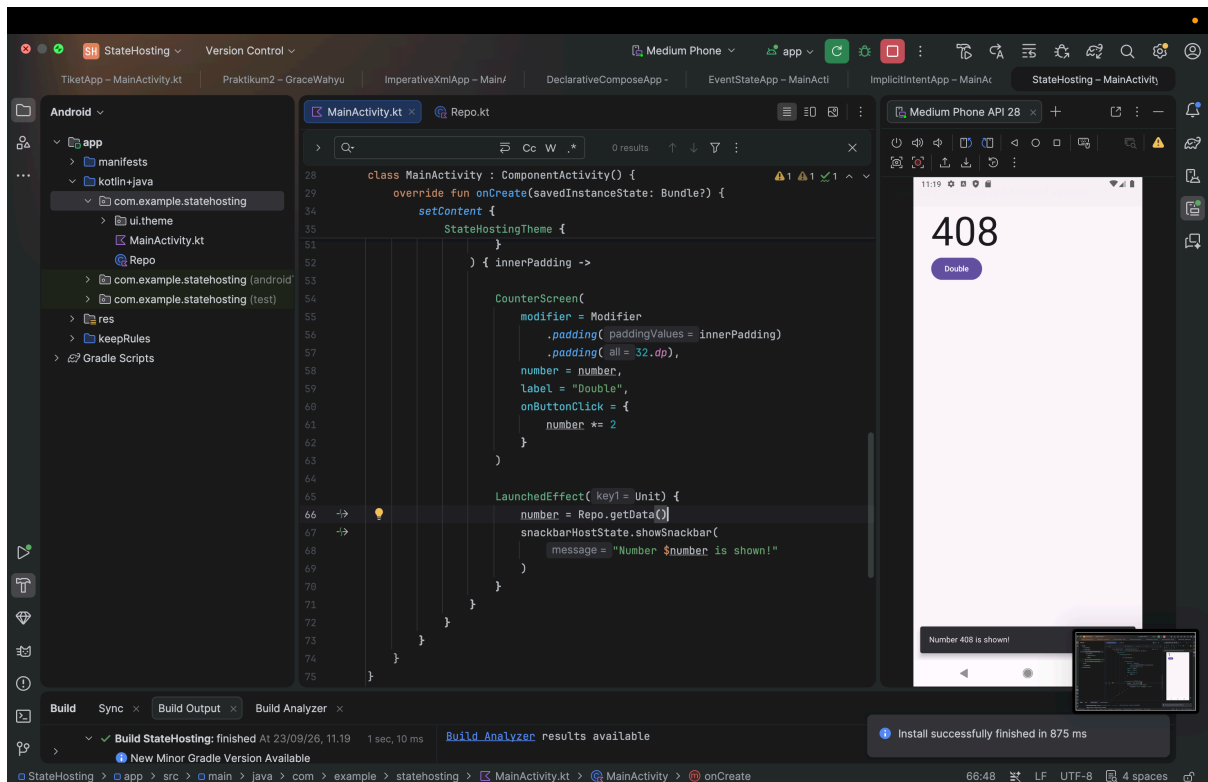
Variabel number digandakan, Text diperbarui, dan Snackbar menampilkan pesan baru yang memuat nilai number terbaru.

D. LaunchedEffect dengan Repo

Repo digunakan untuk mensimulasikan pengambilan data dari jaringan. Fungsi `getData` menunda coroutine selama dua detik dan mengembalikan bilangan acak.



Gambar 16. Kondisi awal sebelum data dari Repo selesai dimuat.

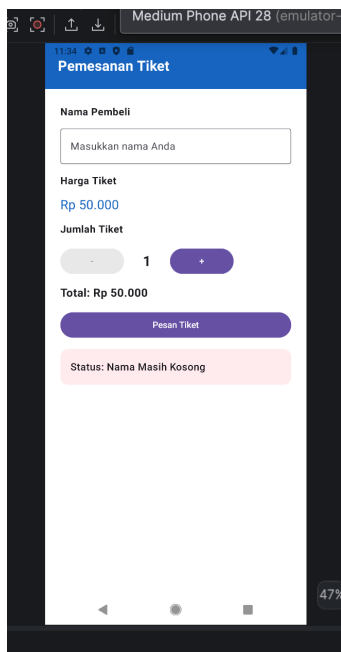


Gambar 17. Nilai berubah menjadi 408 setelah Repo mengembalikan data acak.

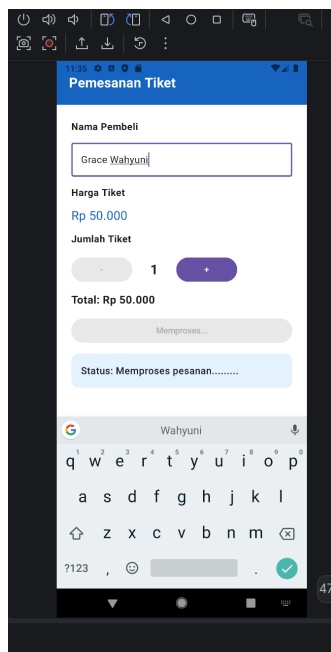
- **Apa yang memicu perubahan angka pada percobaan Repo?**
LaunchedEffect(Unit) dijalankan satu kali ketika composable masuk composition. Setelah delay selesai, hasil Repo.getData() ditulis ke state number sehingga UI melakukan recomposisi.
- **Apa perbedaan LaunchedEffect(number) dan LaunchedEffect(Unit)?**
LaunchedEffect(number) berjalan ulang setiap key number berubah. LaunchedEffect(Unit) dijalankan ketika composable masuk composition dan tidak diulang selama key Unit tetap sama.
- **Kapan masing-masing efek digunakan?**
Kita menggunakan key state untuk efek yang harus mengikuti perubahan state, misalnya menampilkan Snackbar. Gunakan Unit untuk pekerjaan awal satu kali, misalnya memuat data ketika layar dibuka.

E Latihan Soal Halaman Ticket

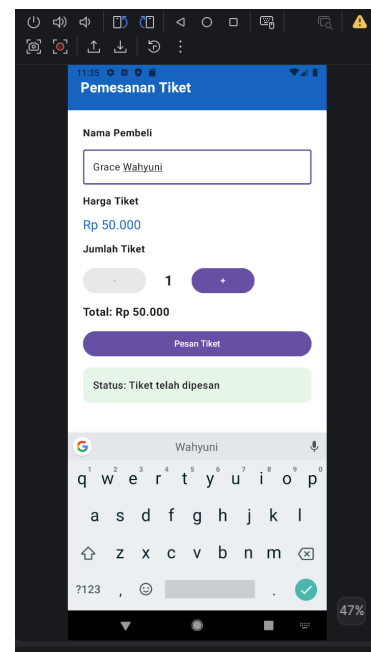
Halaman Ticket menerapkan state hoisting pada halaman pemesanan tiket. Parent menyimpan harga tiket, jumlah tiket, dan nama pembeli, sedangkan child menerima nilai dan callback untuk menampilkan atau mengubah data.



Gambar 18. Kondisi awal dan validasi nama pembeli.



Gambar 19. Status memproses pesanan.



Gambar 20. Status tiket telah dipesan.

- **Bagaimana state hoisting diterapkan pada halaman Ticket?**
Harga tiket, jumlah tiket, dan nama pembeli dikelola oleh TicketPage sebagai fun parent. Kemudian komponen tampilan menerima nilai melalui parameter dan mengirim event melalui callback onNamaChange, onTambahJumlah, onKurangiJumlah, dan onPesanTiket.
- **Bagaimana total harga dihitung?**
Total harga dihitung menggunakan rumus harga Tiket dikalikan jumlahTiket. Ketika jumlah tiket berubah, Compose melakukan recomposition dan total diperbarui otomatis.
- **Bagaimana validasi nama pembeli bekerja?**
Ketika nama kosong, callback pemesanan tidak menjalankan proses pemesanan dan status diubah menjadi Isi nama kamu terlebih dahulu. Ketika nama telah diisi, kemudian proses pemesanan dimulai.
- **Bagaimana urutan status pemesanan?**
Pertama, LaunchedEffect akan dipicu oleh id pemesanan mengubah status menjadi “Memproses pesanan.....”, kemudian kita menunggu lima detik dengan delay(5000), lalu mengubah status menjadi Tiket telah dipesan.
- **Mengapa LaunchedEffect sesuai untuk proses tersebut?**

Karena delay merupakan operasi suspend yang tidak boleh dijalankan langsung di event UI. `LaunchedEffect` menyediakan coroutine yang terikat pada lifecycle composable sehingga proses dapat dibatalkan jika halaman keluar dari composition.

Kesimpulan

Praktikum ini menunjukkan bahwa state hosting memisahkan pengelolaan data dari tampilan. Perubahan konfigurasi dapat ditangani dengan `rememberSaveable` untuk state UI sederhana. `LaunchedEffect` digunakan untuk menjalankan side effect yang bergantung pada perubahan state atau pekerjaan awal satu kali. Ketiga konsep tersebut kita terapkan pada tugas halaman Ticket: parent mengelola state, child merender UI, dan `LaunchedEffect` mengatur alur status pemesanan secara asinkron.